

Rapport de projet

Sujet : Jeu d'échec martien

Auteurs : Yanis Hammaoui 22510983
Rayane Kachbi 22307872

Année universitaire : 2025/2026

Confidentialité : Réservé aux étudiants UFR Maths-Info Paris Cité.

Type de diffusion : Document électronique (.pdf)

Sommaire

1	Introduction	3
2	Présentation du jeu	4
2.1	Règles du jeu	4
2.2	Conditions de victoire	5
2.3	Hypothèses et simplifications	5
3	Implémentation du moteur de jeu	6
3.1	Représentation des états	6
3.2	Génération des coups	6
3.3	Transitions et états terminaux	7
3.4	Validation du moteur	7
4	Intelligence artificielle	8
4.1	Algorithme Minimax	8
4.2	Élagage alpha-bêta	8
4.3	Niveaux de difficulté	9
5	Fonctions d'évaluation	10
5.1	Description des fonctions	10
5.2	Comparaison expérimentale	11
5.3	Impact sur l'algorithme	11
6	Tournoi entre les intelligences artificielles	11
6.1	Protocole expérimental	11
6.2	Résultats	12
6.3	Analyse des résultats	12
6.4	Étude annexe : Impact de la randomisation des ouvertures	14
6.5	Étude annexe : Impact de l'élagage Alpha-Bêta sur les performances	15
7	Utilisation d'outils d'IA générative	16
7.1	Outils utilisés et environnement	16
7.2	Rôle effectif : Planification et compréhension	16
7.3	Génération de code et apprentissage	16
7.4	Analyse critique et limites constatées	17
8	Difficultés rencontrées	17
9	Conclusion	18
A	Annexes : Historique des prompts génératifs	18

Liste des tableaux

1	Impact direct de l'élagage sur le temps de calcul de la difficulté "Difficile".	9
2	Comparaison des performances et temps de calcul des heuristiques.	11
3	Résultats du tournoi déterministe sur 50 parties par couple d'IA.	12
4	Résultats du tournoi avec profondeur égal pour tout niveau de difficulté. .	13
5	Résultats du tournoi avec randomisation des premiers coups (50 parties/- couple).	15
6	Comparaison des temps de réflexion moyens avec et sans élagage.	15

1 Introduction

Dans le cadre de l'Unité d'Enseignement d'Intelligence Artificielle, nous avons réalisé ce projet avec l'objectif de concevoir et d'implémenter un jeu de stratégie déterministe à connaissances parfaites.

Notre choix s'est porté sur le jeu d'échecs martiens (Martian Chess), un jeu de stratégie abstraite inventé en 1997 par Andrew Looney.

Le projet s'articule autour de deux défis majeurs :

- Le premier consiste au développement d'un moteur de jeu robuste en Python, modélisant les règles, la génération des états et les conditions de victoire.
- Le second défi, qui constitue le cœur de ce projet, repose sur l'implémentation de l'algorithme Minimax couplé à l'élagage alpha-bêta.

Vous trouverez dans ce rapport l'ensemble de notre démarche scientifique et technique.

En commençant par une présentation des règles et des spécificités du jeu.

Nous exposerons ensuite les choix d'implémentation de notre moteur de jeu (représentation, transitions, validation), avant de détailler la conception théorique et pratique de nos intelligences artificielles.

Par la suite, une analyse expérimentale reposant sur un tournoi automatisé permettra de comparer les performances et les stratégies de nos IA.

Enfin, nous exposerons tous ce qu'il y a à savoir sur notre utilisation des outils d'IA générative avant de conclure sur les difficultés rencontrées et les solutions apportées.

2 Présentation du jeu

Martian Chess (Échecs Martiens) est un jeu de stratégie inventé par Andrew Looney en 1997. Il se joue traditionnellement avec un ensemble de pyramides appelées "Looney Pyramids". La particularité fondamentale de ce jeu, qui le distingue des échecs classiques, réside dans le fait que l'appartenance des pièces n'est pas définie par leur couleur, mais exclusivement par leur position géographique sur le plateau à un instant T.

2.1 Règles du jeu

Comme illustré sur la Figure 2, le plateau de jeu est constitué d'un damier de 4×8 cases, séparé en son centre par une ligne appelée le "canal". Ce canal délimite les deux zones (de 4×4 cases) appartenant à chaque joueur. Chaque participant commence la partie avec 9 pièces disposées dans un coin de sa zone selon une symétrie rotationnelle.

L'arsenal initial est composé de :

- **3 Reines** : grandes pyramides à 3 points.
- **3 Drones** : moyennes pyramides à 2 points.
- **3 Pions** : petites pyramides à 1 point.

La règle d'or (règle de propriété) stipule qu'un joueur ne peut déplacer que les pièces qui se trouvent physiquement dans sa propre moitié de terrain au moment de jouer. Dès qu'une pièce franchit le canal et atterrit dans la zone adverse, elle change immédiatement de propriétaire.

Les déplacements varient selon le type de pièce et il est formellement interdit de sauter par-dessus une autre pièce :

- **Pion** : Se déplace d'exactly une case, en diagonale.
- **Drone** : Se déplace d'une ou deux cases, orthogonalement (sur les lignes horizontales ou verticales).
- **Reine** : Se déplace d'un nombre illimité de cases dans toutes les directions (orthogonales et diagonales).

Une capture s'effectue lorsqu'une pièce termine son déplacement sur une case occupée par une pièce adverse (la capture implique donc obligatoirement de franchir le canal). La pièce capturée est retirée du plateau et conservée par le joueur pour le décompte final.

Deux règles additionnelles viennent enrichir la tactique :

- **Field Promotions (Fusions)** : Si un joueur ne possède plus de Reine dans sa zone, il peut utiliser son tour pour fusionner un Pion et un Drone (ou l'inverse) adjacents et les remplacer par une Reine. De même, l'absence de Drone permet de fusionner deux Pions.
- **No Undo** : Il est interdit d'annuler le coup précédent de l'adversaire en renvoyant immédiatement la pièce qu'il vient de déplacer vers notre zone sur sa case de départ.

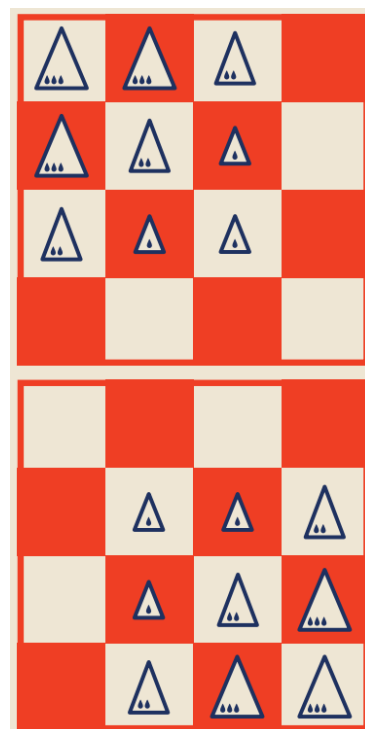


FIGURE 1 – Configuration initiale du plateau.

2.2 Conditions de victoire

L'objectif du jeu est de maximiser son score, qui est calculé en additionnant la valeur des pièces capturées (Reine = 3, Drone = 2, Pion = 1). Une partie se termine de deux manières possibles :

- **Zone vide** : La partie s'arrête instantanément dès qu'un des deux joueurs n'a plus aucune pièce dans sa zone. Le vainqueur est celui qui possède le score le plus élevé. En cas d'égalité des scores, le joueur qui a provoqué la fin de la partie (en vidant une zone) est déclaré vainqueur.
- **Deadlock (Impasse)** : Si la partie stagne, un joueur peut "appeler la pendule" (Call Deadlock). À partir de ce moment, un compte à rebours s'enclenche : si sept tours consécutifs s'écoulent sans qu'aucune capture ne soit réalisée par l'un des joueurs, la partie s'arrête. Les scores sont alors comparés. Si les scores sont égaux dans ce cas de figure, la partie se solde par un match nul.

2.3 Hypothèses et simplifications

Afin de modéliser le jeu informatiquement, nous avons adopté les choix et simplifications suivants :

- **Mode 2 joueurs exclusif** : Bien que les règles originales évoquent une variante à 4 joueurs, notre implémentation a été strictement restreinte au mode duel opposant le Joueur 0 (Haut) au Joueur 1 (Bas), conformément aux exigences d'un jeu à deux joueurs à somme nulle.
- **Abstraction des couleurs** : Les couleurs des pièces physiques, destinées à perturber visuellement les joueurs humains, ont été complètement ignorées dans notre modèle de données (structure `GameState`). L'appartenance d'une pièce est évaluée mathématiquement en fonction de son index de ligne par rapport à la constante `MIDLINE`.
- **Utilisation du Deadlock par les IA** : Le mécanisme de *Deadlock*, a nécessité une adaptation spécifique pour les intelligences artificielles.

Dans une première version, les IA déclenchaient systématiquement le Deadlock dès qu'elles obtenaient un avantage au score. Cependant, ce comportement produisait des parties artificielles et peu réalistes, car le Deadlock est censé être utilisé uniquement dans des situations d'impasse ou de répétition de coups.

Nous avons donc introduit une contrainte comportementale supplémentaire : une IA ne peut appeler le Deadlock que si un certain nombre de coups sans capture a déjà été atteint. Dans notre implémentation finale, l'activation devient possible après 10 coups sans capture.

Par ailleurs, afin d'éviter des parties infinies lorsque aucun joueur ne prend l'avantage, une limite supplémentaire de 30 coups sans capture entraîne automatiquement l'appel du Deadlock.

3 Implémentation du moteur de jeu

La modélisation informatique de Martian Chess a été réalisée en Python, en adoptant une architecture modulaire séparant la logique métier (`rules.py` et `model.py`), l'interface graphique (`view.py`) et l'intelligence artificielle (`minimax.py` et `heuristics.py`).

3.1 Représentation des états

L'état complet d'une partie à un instant T est encapsulé dans la classe `GameState` (dans `model.py`). Le plateau de jeu est modélisé par une liste de listes (8 lignes pour 4 colonnes). Chaque cellule contient soit un espace vide ("."), soit une chaîne de caractères représentant une pièce ("P" pour Pawn, "D" pour Drone, "Q" pour Queen).

Comme la notion de "couleur des pièces" ne compte pas dans ce jeu, l'appartenance d'une pièce est calculée dynamiquement via la fonction `is_own_side` (dans `rules.py`), qui vérifie simplement si l'index de la ligne se situe au-dessus ou en dessous de la ligne médiane (`MIDLINE = 4`).

Outre le plateau, l'objet `GameState` conserve en mémoire toutes les variables nécessaires à l'évaluation du jeu :

- `scores` : Une liste de deux entiers stockant les points des joueurs.
- `captured_pieces` : Un dictionnaire répertoriant le type des pièces capturées pour l'affichage de l'inventaire.
- `current_player` : L'identifiant (0 ou 1) du joueur dont c'est le tour.
- `last_move` : Un tuple gardant une trace du dernier coup joué.
- `moves_without_capture` et `deadlock_active` : Variables gérant la règle de la pendule.

Enfin, la classe implémente une méthode `copy()` permettant d'effectuer une copie profonde de l'état. Cette méthode permet à l'algorithme Minimax de simuler des milliers de coups en mémoire sans altérer le véritable plateau affiché à l'écran.

3.2 Génération des coups

La génération des coups légaux s'effectue en deux temps. Dans un premier temps, la méthode `get_legal_moves` parcourt le plateau et identifie toutes les pièces se trouvant dans la zone du joueur courant. Pour chacune d'elles, la fonction `get_piece_moves` calcule les destinations possibles :

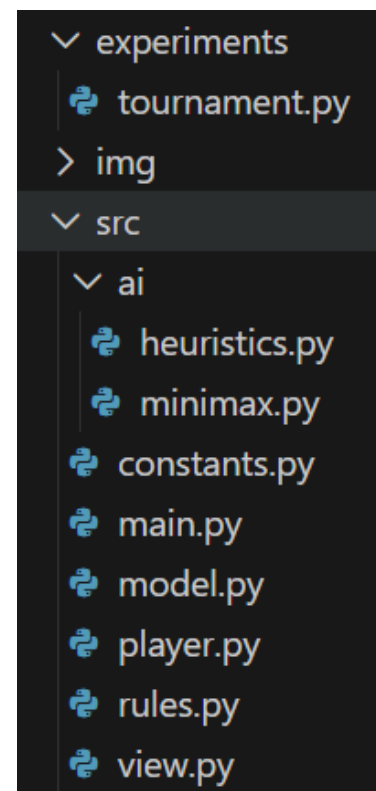


FIGURE 2 – Arborescence du code source du projet.

- **Mouvements classiques** : Des boucles explorent les directions propres à chaque pièce. Pour les Drones et les Reines, l'exploration d'une direction s'interrompt si une autre pièce bloque le passage (interdiction de sauter).
- **Fusion de pièces** : La fonction vérifie via `count_player` si le joueur est en pénurie d'un type de pièce (Drone ou Reine) et si la réserve globale le permet (`piece_available`). Si les conditions sont remplies, les fusions possibles entre pièces adjacentes sont générées sous la forme d'un tuple spécifique de 6 éléments (ex : ("fusion", r1, c1, r2, c2, RESULT_PIECE)).

Dans un second temps, la méthode `get_legal_moves` applique la règle stricte du "No Undo". Elle génère le mouvement inverse de `last_move` et le retire de la liste des coups légaux, empêchant ainsi le joueur de renvoyer immédiatement une pièce reçue sur sa case d'origine.

3.3 Transitions et états terminaux

Lorsqu'un coup est sélectionné, la méthode `apply_move` met à jour l'état du jeu. Elle distingue les déplacements classiques (tuple de taille 4) des fusions (tuple de taille 6).

Lors d'un déplacement classique, si la case d'arrivée n'est pas vide, une capture est détectée : le score du joueur courant est incrémenté de la valeur de la pièce (`PIECE_VALUES`), et le compteur `moves_without_capture` est réinitialisé à zéro.

Dans le cas contraire, ce compteur est incrémenté. À la fin de la fonction, la pièce est déplacée dans la matrice et le tour passe à l'adversaire.

La détection de fin de partie (`is_terminal`) s'active si l'une des deux conditions suivantes est remplie :

- L'un des deux joueurs n'a plus aucune pièce dans sa zone (vérifié par la méthode `has_pieces`).
- Le mode deadlock a été activé (`deadlock_active = True`) et le seuil des 7 tours sans capture est atteint.

Si l'état est terminal, la fonction `get_winner` détermine le vainqueur. En cas d'égalité aux points sans deadlock, le code attribue la victoire au joueur ayant déclenché la fin de partie.

Puisque `apply_move` change de joueur à la toute fin de son exécution, le joueur ayant validé le coup final est devenu l'adversaire de `current_player` (`1 - current_player`).

3.4 Validation du moteur

L'intégration de Pygame (`view.py`) nous a permis de valider la bonne génération des coups en affichant des indicateurs visuels (points bleus) sur les cases de destination légales lors de la sélection d'une pièce à la souris.

Cette interface graphique a facilité les tests d'assurance qualité lors d'affrontements "Humain contre Humain", validant ainsi le bon déclenchement des fusions (seulement quand la pièce manque à l'appel) et l'interdiction de l'Undo.

De plus, nous avons pris le temps de tester manuellement l'intégralité des cas de figure possibles. À chaque étape du développement, dès qu'une nouvelle règle était implémentée, elle faisait l'objet d'un test systématique pour vérifier qu'aucun comportement incohérent ne venait perturber l'état du jeu.

4 Intelligence artificielle

4.1 Algorithme Minimax

Dans notre code (`minimax.py`), la fonction `minimax` prend en paramètres :

- L'état courant.
- La profondeur restante (`depth`).
- Un booléen indiquant si l'on cherche à maximiser le score (`maximizing_player`).
- L'ID du joueur racine.
- La fonction d'évaluation à utiliser (`eval_func`).

La condition d'arrêt s'enclenche lorsque la profondeur atteint 0 ou que l'état est terminal. Dans un état terminal, l'algorithme renvoie l'infini positif (victoire), l'infini négatif (défaite) ou 0 (match nul). Si l'arrêt est dû à la limite de profondeur, la fonction d'évaluation est appelée.

Pour la simulation des coups, l'algorithme utilise la méthode `state.copy()` développée dans la section précédente, ce qui lui permet d'appliquer virtuellement chaque coup de la liste générée par `get_legal_moves` sur un état cloné, sans affecter la partie en cours.

4.2 Élagage alpha-bêta

En milieu de partie, le facteur de branchement est élevé. Pour y remédier, nous avons implémenté l'élagage alpha-bêta au sein de la fonction `alpha_beta`.

En plus des paramètres de minmax, nous avons introduit les deux paramètres suivants :

- α (**alpha**) : Le meilleur score (maximum) que l'IA est déjà assurée d'obtenir.
- β (**bêta**) : Le pire score (minimum) que l'adversaire est déjà assuré d'imposer.

Lors de l'exploration, la condition `if beta <= alpha: break` est vérifiée à chaque itération. Si l'adversaire constate qu'une branche offre à l'IA une opportunité supérieure à un autre coup déjà évalué, il ne choisira jamais de s'y engager. L'algorithme interrompt alors immédiatement l'exploration de cette sous-branche.

Bien sûr, l'intégration de cet élagage n'altère en rien la décision finale de l'IA (le résultat est strictement identique au Minimax), mais elle réduit drastiquement le nombre de nœuds explorés, permettant ainsi d'augmenter la profondeur de recherche pour un temps de calcul équivalent.

Algorithme (Profondeur 4)	Temps moyen par coup
Minimax pur	≈ 15.67 s
Minimax avec Alpha-Bêta	≈ 2.90 s

TABLE 1 – Impact direct de l'élagage sur le temps de calcul de la difficulté "Difficile".

4.3 Niveaux de difficulté

Afin d'offrir une progression adaptée aux joueurs humains et de préparer nos expérimentations, nous avons structuré la classe `AIPlayer` (`player.py`) en trois niveaux de difficulté distincts. Ces niveaux se différencient par la profondeur de l'arbre exploré et par la complexité de leur compréhension du jeu (fonctions d'évaluation) :

- **IA Facile (Niveau 1)** : Configurée avec une profondeur faible ($depth = 2$) et couplée à la fonction d'évaluation `eval_score_only`. L'IA ne prévoit que son coup immédiat et la réponse directe de l'adversaire, elle se base exclusivement sur la différence de points déjà accumulés.

Si elle peut capturer une Reine, elle le fera systématiquement, même si cela donne le contrôle d'une de ses propres pièces vitales à l'adversaire au coup suivant.

- **IA Moyenne (Niveau 2)** : Utilise une profondeur modérée ($depth = 3$) et la fonction `eval_positional`. Elle anticipe mieux les réponses adverses et prend en compte le rapport de force matériel global sur le plateau.

Elle utilise une évaluation matérielle (la fonction `eval_material`) afin de comparer la valeur totale des pièces présentes dans chaque camp, et privilégie les positions où elle possède un avantage matériel.

Elle privilégie le contrôle de la zone centrale du plateau afin de maintenir une pression sur l'adversaire.

En fin de partie, son comportement s'adapte au score : si elle mène, elle cherchera à vider rapidement sa zone pour provoquer la fin de partie ; si elle perd, elle tentera au contraire de conserver des pièces pour prolonger le jeu.

Enfin, elle favorise les positions offrant une bonne mobilité tout en limitant les possibilités d'action adverses.

- **IA Difficile (Niveau 3)** : Atteint une profondeur importante ($depth = 4$) et utilise l'heuristique avancée `eval_pression`.

Elle privilégie les positions offrant un fort potentiel de capture afin de maintenir une pression constante sur l'adversaire.

L'évaluation prend en compte à la fois le score actuel et les menaces immédiates présentes sur le plateau. Une position permettant de capturer des pièces adverses sera fortement valorisée, tandis qu'une position laissant trop d'opportunités de capture à l'adversaire sera pénalisée.

Stratégie de Deadlock autonome :

Indépendamment de leur niveau, toutes nos IA intègrent une logique stratégique globale dans leur méthode `get_move`.

Avant même de lancer l'exploration Alpha-Bêta, l'IA analyse les scores actuels, et si elle constate qu'elle mène aux points et que la pendule n'est pas activée et qu'un certain nombre de coups dans capture a été atteint (10 coups) elle déclenchera automatiquement l'appel au "Deadlock" via `state.enable_deadlock()`.

Par ailleurs, lorsqu'aucun joueur ne parvient à prendre un avantage clair, un appel automatique au *Deadlock* est effectué après 30 coups sans capture afin d'éviter les parties infinies et les situations de répétition.

Ce mécanisme vise à reproduire un comportement plus réaliste et équitable : le *Deadlock* n'est pas utilisé comme une stratégie abusive pour gagner immédiatement, mais comme un outil de gestion des situations d'impasse, similaire à ce que ferait un joueur humain.

5 Fonctions d'évaluation

5.1 Description des fonctions

Nous avons implémenté trois fonctions dans `heuristics.py`, chacune correspondant à un niveau de compréhension du jeu différent :

- **eval_score_only (IA Facile)** : Elle se base uniquement sur la différence de score immédiate (`my_score - opponent_score`). Cette fonction ignore totalement la configuration des pièces sur le plateau.
- **eval_material (IA Moyenne)** : Elle intègre la valeur des pièces présentes dans la zone du joueur (`PIECE_VALUES`). Le score de capture est toutefois priorisé (multiplié par 100) afin de garantir qu'une capture réelle soit toujours considérée comme plus importante qu'un simple avantage positionnel.
- **eval_positionall (IA Moyenne)** : Cette fonction reprend l'évaluation matérielle de `eval_material`, en conservant la priorité donnée au score de capture, puis y ajoute une dimension stratégique. Elle valorise le contrôle de la zone centrale du plateau, pénalise les positions favorables à l'adversaire et adapte son comportement à la fin de partie : vider rapidement sa zone lorsqu'elle mène au score ou conserver des pièces lorsqu'elle est en retard. Elle prend également en compte la mobilité afin de favoriser les positions offrant davantage de coups possibles.
- **eval_pression (IA Difficile)** : Cette heuristique adopte une approche plus agressive orientée vers le potentiel offensif immédiat. En plus du score officiel actuel, elle évalue la pression exercée sur l'adversaire en analysant tous les coups de capture disponibles pour les deux joueurs.

Pour chaque capture possible, un bonus proportionnel à la valeur de la pièce menacée est ajouté (`PIECE_VALUES[target] * 10`). L'heuristique calcule ensuite une différence de pression offensive entre les deux camps :

```
pressure_diff = attack_score - opp_attack_score
```

5.2 Comparaison expérimentale

L'analyse des fichiers `results.csv` nous permet de comparer l'efficacité de ces approches.

Niveau d'IA	Temps moyen par coup	Performance (vs Facile)
IA Facile (p=2)	$\approx 0,005$ s	-
IA Moyenne (p=3)	≈ 0.284 s	100 % de victoires
IA Difficile (p=4)	≈ 1.851 s	100 % de victoires

TABLE 2 – Comparaison des performances et temps de calcul des heuristiques.

On observe que :

1. **Complexité vs Efficacité** : L'IA Difficile met environ 6,5 fois plus de temps à réfléchir que l'IA Moyenne (passant de 0,28 s à 1,85 s en moyenne), mais cela se traduit par une domination totale sur le plateau.
2. **Rapidité de la fonction Facile** : Avec 0,005 s par coup, l'heuristique `eval_score_only` est extrêmement rapide mais totalement inefficace, car elle ne voit pas les menaces matérielles imminentes.

5.3 Impact sur l'algorithme

La précision de la fonction d'évaluation a bien évidemment un impact direct sur l'efficacité de l'élagage alpha-bêta.

Une fonction plus intelligente comme `eval_positional` permet d'attribuer des scores très différenciés aux branches de l'arbre.

En identifiant rapidement les mauvais coups (comme laisser une Reine être capturée), l'IA crée des décalages importants entre les valeurs α et β .

Cela permet à l'IA Difficile d'atteindre une profondeur de 4 en seulement 1,85 seconde malgré l'augmentation exponentielle du nombre de nœuds.

6 Tournoi entre les intelligences artificielles

Afin d'évaluer les performances, la robustesse et les temps de calcul de nos trois niveaux d'Intelligence Artificielle, nous avons mis en place un protocole de tournoi automatisé (via le script `tournament.py`).

6.1 Protocole expérimental

Le tournoi a fait s'affronter chaque paire d'IA possible (Facile vs Moyen, Facile vs Difficile, etc.), y compris des matchs miroirs (Facile vs Facile). Pour chaque confrontation, le protocole suivant a été appliqué :

- **Nombre de parties** : 50 matchs par couple.

- **Équité du premier joueur** : Afin d'annuler un éventuel avantage au joueur commençant la partie, l'alternance a été forcée : l'IA 1 a commencé les 25 premières parties, et l'IA 2 a commencé les 25 suivantes.
- **Déterminisme strict** : Conformément aux consignes du projet, aucune part de hasard (randomisation des coups à score égal ou ouvertures aléatoires) n'a été introduite dans les choix de l'IA.
- **Métriques récoltées** : Nombre de victoires pour chaque IA, nombre de matchs nuls (déclenchés par la limite de Deadlock), temps moyen d'une partie et temps moyen de réflexion par coup pour chaque IA.

6.2 Résultats

Les données compilées à l'issue de l'exécution du tournoi sont présentées dans le Tableau 3 ci-dessous.

Confrontation	Vic. IA 1	Vic. IA 2	Nuls	Temps/Partie	Temps/Coup (IA 1)	Temps/Coup (IA 2)
Facile vs Facile	25	25	0	0.39 s	0.0050 s	0.0051 s
Facile vs Moyen	0	50	0	7.11 s	0.0061 s	0.2839 s
Facile vs Difficile	0	50	0	44.55 s	0.0053 s	1.8510 s
Moyen vs Moyen	25	25	0	9.64 s	0.2188 s	0.2195 s
Moyen vs Difficile	0	50	0	74.05 s	0.2510 s	2.9000 s
Difficile vs Difficile	25	25	0	121.52 s	1.9266 s	1.9311 s

TABLE 3 – Résultats du tournoi déterministe sur 50 parties par couple d'IA.

6.3 Analyse des résultats

L'interprétation de ce tableau met en lumière plusieurs phénomènes inhérents à l'algorithme Minimax et à nos heuristiques :

1. Une hiérarchie de niveau absolue et incontestable

Les résultats montrent une domination parfaite (taux de victoire de 100 %) des IA de niveau supérieur sur les IA de niveau inférieur.

L'IA Difficile a remporté ses 50 matchs face à l'IA Moyenne, qui elle-même a écrasé l'IA Facile 50 à 0. Cette asymétrie prouve que l'augmentation de la profondeur (de 2 à 4) couplée à une heuristique intégrant la position et la gestion de la fin de partie (eval_positional) surpasse totalement une simple recherche de capture matérielle.

2. Le phénomène du 25/25 et le strict déterminisme

Lors des matchs miroirs (ex : Moyen vs Moyen), le résultat affiche une égalité parfaite de 25 victoires partout, sans aucun match nul. Il est crucial de comprendre qu'il ne s'agit pas d'un équilibre probabiliste (une chance sur deux de gagner), mais d'une conséquence directe du déterminisme du jeu.

Puisqu'aucun hasard n'est implémenté, deux IA identiques confrontées à l'état initial du plateau joueront toujours exactement la même suite de coups. Le premier joueur remporte donc la partie déterministe.

Puisque l'IA 1 commence exactement 25 fois, elle remporte exactement 25 victoires. Lorsque l'on inverse les rôles pour les 25 parties suivantes, c'est l'IA 2 (qui commence) qui remporte les 25 parties. Cela démontre l'existence d'un fort "avantage au premier joueur" à *Martian Chess* à profondeur de calcul égale.

3. Évolution des temps de calcul

Le temps moyen par coup illustre clairement la croissance de la complexité temporelle du Minimax, en accord avec une complexité exponentielle $O(b^d)$, où b représente le facteur de branchement et d la profondeur de recherche :

- L'IA Facile (profondeur 2) reste très rapide, avec un temps moyen d'environ 5 millisecondes par coup.
- L'IA Moyenne (profondeur 3) montre une augmentation notable du coût de calcul, avec environ 250 à 280 millisecondes par coup.
- L'IA Difficile (profondeur 4) atteint un temps moyen compris entre 1.9 et 2.9 secondes par coup.

On observe ainsi une augmentation très significative du temps de calcul entre chaque niveau de profondeur, notamment un facteur proche de 10 entre l'IA Moyenne et l'IA Difficile.

Malgré cette croissance, l'implémentation avec élagage alpha-bêta permet de maintenir des temps de calcul exploitables en pratique, même à profondeur 4, ce qui rend l'IA Difficile encore jouable dans un contexte interactif.

Enfin, ces résultats confirment empiriquement l'impact du facteur de branchement élevé du jeu, avec un temps de résolution fortement dépendant de la complexité de l'arbre de recherche.

4. Tournoi a profondeur égal

En plus du tournoi précédent nous avons réaliser un deuxième tournoi en suivant le même protocol, cependant nous avons mis la profondeur de tous les niveaux de difficulté a 2, afin d'évaluer les fonctions d'évaluation indépendamment de la profondeur de l'arbre de recherche et nous avons obtenue les suivant (fichier `results_profondeur_egal.csv`)

Confrontation	Vic. IA 1	Vic. IA 2	Nuls	Temps/Partie	Temps/Coup (IA 1)	Temps/Coup (IA 2)
Facile vs Moyen	0	50	0	0.68 s	0.0061 s	0.0216 s
Facile vs Difficile	0	50	0	0.66 s	0.0053 s	0.0190 s
Moyen vs Difficile	0	50	0	1.45 s	0.0266 s	0.0411 s

TABLE 4 – Résultats du tournoi avec profondeur égal pour tout niveau de difficulté.

Les résultats montrent que, indépendamment de la profondeur de recherche, les IA de niveau supérieur dominent systématiquement les IA plus faibles. Cela met en évidence

l'impact déterminant des fonctions d'évaluation, dont la qualité et la sophistication augmentent avec le niveau de difficulté.

5. Facteur de branchement

Nous avons également profité des tournois pour estimer le facteur de branchement du jeu. Pour cela, nous avons calculé la moyenne du nombre de coups possibles à chaque position exploré. Nous obtenons ainsi un facteur de branchement d'environ 20. Les résultats détaillés sont disponibles dans le fichier `results.csv`.

6. Remarques

Initialement, l'IA de niveau moyen utilisait la fonction `eval_material` et l'IA de niveau difficile utilisait la fonction `eval_positional`. Cependant, l'évaluation basée uniquement sur le matériel n'est pas optimale dans notre cas. Nous avons donc envisagé une évaluation basée sur les menaces de captures. Après plusieurs tests, cette évaluation, utilisée seule et sans ajout supplémentaire, s'est révélée être la meilleure. Nous l'avons donc conservée pour le niveau difficile et avons attribué `eval_positional` au niveau moyen.

À noter que ce changement a augmenté le temps moyen par coup, passant d'un intervalle de 1.2 à 1.6 seconde(s) à un intervalle de 1.9 à 2.9 secondes.

Nous acceptons cette augmentation du temps de calcul, car cette nouvelle fonction d'évaluation est nettement plus cohérente et améliore significativement la qualité des décisions de l'IA.

6.4 Étude annexe : Impact de la randomisation des ouvertures

Afin de confirmer que le phénomène de symétrie (25/25) était exclusivement dû au déterminisme du premier coup, nous avons mené une expérience annexe (hors consigne) en introduisant une randomisation contrainte sur les deux premiers coups joués.

Cette randomisation ne consistait toutefois pas à sélectionner un coup totalement aléatoire parmi tous les coups légaux possibles. Le choix était effectué uniquement parmi les coups considérés comme les meilleurs par l'évaluation de l'IA, afin de conserver un comportement stratégique cohérent tout en diversifiant légèrement les ouvertures possibles.

Cette modification a ensuite été retirée pour respecter la consigne, mais les résultats obtenus (présentés dans le Tableau 5) offrent des perspectives intéressantes.

Confrontation	Vic. IA 1	Vic. IA 2	Nuls
Facile vs Facile	28	22	0
Facile vs Moyen	2	48	0
Facile vs Difficile	0	49	1
Moyen vs Moyen	26	23	1
Moyen vs Difficile	17	26	7
Difficile vs Difficile	25	25	0

TABLE 5 – Résultats du tournoi avec randomisation des premiers coups (50 parties/-couple).

La lecture de ces données met en évidence la robustesse de nos algorithmes face à l'imprévu :

- **Bris de la symétrie** : Le ratio parfait de 25/25 disparaît totalement dans les matchs miroirs, prouvant que les parties générées sont désormais uniques.
- **Maintien de la hiérarchie** : Malgré des ouvertures non-optimales imposées par l'aléatoire, la hiérarchie des IA est conservée. L'IA Difficile s'adapte et bat l'IA Moyenne à dans la majorité des cas. Les rares victoires des IA de niveau inférieur s'expliquent par le fait que le coup aléatoire initial a pu exceptionnellement placer l'IA supérieure dans une situation irrécupérable.
- **Apparition des matchs nuls** : L'ouverture aléatoire empêchant les IA de dérouler leur "partie parfaite", nous observons l'apparition de résultats "Nuls". Le match *Difficile vs Moyen* culmine avec 7 matchs nuls, témoignant d'affrontements beaucoup plus disputés où les deux IA parviennent à se neutraliser mutuellement.

6.5 Étude annexe : Impact de l'élagage Alpha-Bêta sur les performances

Pour quantifier précisément l'apport de l'élagage Alpha-Bêta, nous avons relancé notre protocole de tournoi (sur des échantillons de 5 parties) en utilisant directement le Minimax pur, explorant l'intégralité de l'arbre des possibles.

Le Tableau 6 met en évidence la différence de temps de calcul par coup entre les deux approches en fonction de la profondeur.

Niveau de l'IA	Temps/Coup (Minimax pur)	Temps/Coup (Alpha-Bêta)	Facteur d'accélération
Facile (Prof. 2)	0.003 s	0.004 s	Similaire
Moyen (Prof. 3)	0.162 s	0.284 s	Similaire
Difficile (Prof. 4)	15.675 s	1.851 s	≈ 8x plus rapide

TABLE 6 – Comparaison des temps de réflexion moyens avec et sans élagage.

Analyse des performances :

- **Aux profondeurs 2 et 3 :** Le Minimax se révèle aussi rapide, voire très légèrement plus performant que l'Alpha-Bêta.

Cela s'explique par le fait qu'à faible profondeur, l'arbre reste de taille modeste.

Le surcoût processeur lié à la gestion des variables α et β , ainsi qu'aux tests de coupures de branches à chaque itération, n'est pas encore rentabilisé par le nombre de nœuds ignorés.

- **A profondeur 4 :** Avec un facteur de branchement de 21, l'arbre de recherche à profondeur 4 contient environ $21^4 \approx 194\,000$ nœuds.

À ce stade, le Minimax nécessite en moyenne 15.67 secondes par coup, ce qui rendrait l'expérience de jeu lente et frustrante face à un humain, l'algorithme Alpha-Bêta parvient à diviser ce temps de calcul par plus de 8 prouvant sa nécessité.

7 Utilisation d'outils d'IA générative

7.1 Outils utilisés et environnement

Nous avons principalement utilisé **Google Gemini** tout au long du projet, en le complétant ponctuellement par **Claude** (via leurs interfaces web respectives) afin de comparer leurs propositions d'implémentation algorithmique.

L'utilisation s'est faite exclusivement sous la forme d'un assistant conversationnel externe, sans intégration de complétion automatique directe dans notre éditeur de code. Des exemples concrets de prompts utilisés sont consultables dans la partie annexes de ce document (voir Section A).

7.2 Rôle effectif : Planification et compréhension

L'IA a joué un rôle crucial dans la phase de conception du projet.

Le jeu Martian Chess nous étant inconnu, nous avons utilisé Gemini pour synthétiser les règles à partir d'une vidéo YouTube via un prompt détaillé (voir Figure 4). Cela nous a permis de générer rapidement des fiches explicatives claires que nous avons pu nous partager au sein du binôme pour nous approprier la logique du jeu.

De plus, l'IA a servi d'assistant de gestion de projet. En lui fournissant le PDF des consignes, nous avons pu extraire une liste de tâches exhaustives (voir Figure 3) et générer une première proposition d'arborescence de fichiers (voir Figure 5), nous assurant ainsi de ne rater aucun livrable.

7.3 Génération de code et apprentissage

Notre approche du codage a été hybride et itérative :

- **Création de squelettes** : Nous avons demandé à l'IA de générer uniquement les signatures des fonctions nécessaires dans chaque fichier, accompagnées de commentaires explicatifs (voir Figure 6), sans fournir le code interne.
- **Apprentissage accéléré** : L'apport le plus significatif de l'IA a été la découverte de la bibliothèque `Pygame`, que nous ne maîtrisions pas. Au lieu de passer des heures dans la documentation, l'IA nous a fourni des exemples de base clairs, ce qui nous a permis de générer le code du visuel et de vérifier sa robustesse.
- **Implémentation et ajout de fonctionnalités** : Certaines fonctions logiques ont été écrites manuellement puis soumises à l'IA pour vérification ou optimisation. D'autres ont été générées par l'IA, comme l'ajout du bouton "Deadlock" interactif (voir Figure 7).

7.4 Analyse critique et limites constatées

L'utilisation de l'IA générative n'a pas été une solution "magique", et nous avons été confrontés à deux limites majeures qui ont nécessité une certaine prise de recul :

1. Le faux gain de temps sur le code complexe Bien que la génération de code soit instantanée, nous avons constaté que le temps nécessaire pour lire, comprendre en profondeur, adapter à notre architecture et tester le code généré était équivalent au temps qu'il nous aurait fallu pour l'écrire nous-mêmes. Tester minutieusement chaque bloc pour éviter le "code mort" ou les effets de bord a exigé une grande rigueur.

2. Incompréhension des règles spécifiques du jeu Martian Chess n'étant pas un jeu "classique" (comme les échecs ou le morpion), l'IA avait énormément de mal à intégrer la notion d'appartenance géographique des pièces. Lorsque nous lui avons demandé des idées pour concevoir nos fonctions d'évaluation (heuristiques), ses propositions étaient souvent hors-sujet ou démontraient qu'elle avait "oublié" que les pièces changeaient de camp en franchissant le canal.

L'idée des trois heuristiques (Score brut, Matériel, Positionnel avec bonus de fin de partie) a été conçue intégralement par notre binôme. Face à des hallucinations tactiques de la part de l'IA générative, nous avons délibérément choisi de rejeter ses propositions pour les niveaux de difficulté.

8 Difficultés rencontrées

Au cours de la réalisation de ce projet, nous avons été confrontés à plusieurs défis, allant de la conception théorique à l'implémentation technique.

- **Choix d'un jeu original** : La première difficulté a été la sélection du jeu. Conformément aux attentes de l'évaluation valorisant l'originalité, nous souhaitions éviter les classiques (Morpion, Puissance 4, Dames).

Trouver un jeu original, à la fois complexe stratégiquement mais réalisable dans le temps imparti, a demandé quelques jours de recherche.

- **Comportements cycliques et parties infinies** : Lors des premiers affrontements entre nos IA, nous avons observé de nombreuses parties répétitives ou infinies.

L'IA avait tendance à faire des allers-retours sans oser capturer. Pour résoudre ce problème, nous avons modifié nos fonctions d'évaluation pour rendre l'IA plus "agressive" (notamment en multipliant par 100 le score matériel) et en valorisant le contrôle du centre, afin de forcer les interactions.

9 Conclusion

Ce projet d'Intelligence Artificielle a été une expérience enrichissante. Il nous a permis de comprendre un aspect important des jeux vidéo, ce qui change grandement notre vision des choses sur ces IA contre lesquelles nous jouons depuis tout petits.

L'objectif principal a été atteint avec succès : nous avons développé un moteur de jeu robuste et programmé trois niveaux d'Intelligence Artificielle.

L'intégration de l'élagage **alpha-bêta** s'est avérée redoutablement efficace, et la conception de nos fonctions d'évaluation, intégrant la notion de territoire propre à ce jeu, fut un challenge relevé avec succès.

A Annexes : Historique des prompts génératifs

Cette annexe regroupe quelques prompts utilisés avec les outils d'IA générative que l'on a gardé au moment où l'on travaillait sur le projet.

A.1 Phase de planification et compréhension du jeu

*« Dans le cadre du module IA nous avons reçu des consignes pour un projet, voici le pdf avec toutes les instructions.
Je veux que tu me listes tous les éléments à rendre (code, documents), et pour chaque truc, lister aussi les conditions présentés. Le but est d'organiser d'une façon claire toutes les tâches à accomplir, sans rater quoi que ce soit. »*

FIGURE 3 – Prompt utilisé pour l'extraction de la Todo-list depuis le PDF de consignes.

« nous avons décidé de choisir le jeu présenté dans cette vidéo :
["https://youtu.be/AQa6KIQwuj4?si=3QBSbKWHLj7tgncC"](https://youtu.be/AQa6KIQwuj4?si=3QBSbKWHLj7tgncC)
 J'aimerais que tu me rédige ce qui suit :

1. Une courte présentation du jeu.
2. Les éléments nécessaires pour jouer (plateau, les différentes pièces).
3. Une liste de toutes les règles du jeu
4. Les conditions nécessaires pour qu'il y ait un gagnant, un perdant, ou un match nul si c'est possible
5. Le contenu important qui n'a pas été traité dans ma demande, si tout à été traité ce n'est pas nécessaire »

FIGURE 4 – Prompt utilisé pour la synthèse des règles du jeu Martian Chess.

A.2 Phase d'architecture et de génération de code

« on a décidé de coder le jeu en Python. Mon but pour commencer :
 Faire une interface super simpliste du style de l'image jointe.
 pouvoir jouer en 1 contre 1, donc coder le jeu en gros.
 Ce que j'attends de toi :
 D'après tout ce qui est demandé dans le document du projet qui concerne le code,
 fournis moi une structure de fichiers complète (l'arbre) pour l'organisation du
 dossier du projet.
 Quel code se retrouveras dans quel fichier, fournis moi donc une liste de fonctions
 à coder pour chaque partie.
 Aucun code n'est demandé. »

FIGURE 5 – Prompt utilisé pour générer l'arborescence des fichiers du projet.

« Je veux que tu m'écrive en python les définitions des fonctions ainsi que tout ce
 qu'il y a à définir d'autre, et pour le contenu de la fonction met en commentaires
 l'explication ou l'utilité de la fonction sans écrire son code.
 je veux que tu fasse cela pour model.py, view.py, rules.py, main.py »

FIGURE 6 – Prompt utilisé pour la création des squelettes de fonctions (sans implémentation logique).

A.3 Phase d'itération et d'ajout de fonctionnalités

*« finalement, pour le deadlock ce ne sera pas automatique, nous allons rajouter un bouton "Deadlock" pour les deux joueurs en dessous de la liste des pièces capturés (en bas à droite de la fenêtre), si un joueur appuie dessus, un compteur de déplacement sans prise de pièce apparaît, "5 coups avant deadlock", il s'actualise à chaque coup sans prise.
quels sont les fonctions de que je devrai modifier en ajoutant cette règle»*

FIGURE 7 – Prompt ciblé utilisé pour identifier les points d'ancrage d'une nouvelle fonctionnalité dans une architecture existante.